

A REALLY GOOD TITLE

Paxton Fitzpatrick

April 22, 2019

1 Introduction

2 Methods

2.1 Experimental design and data collection

The experiment was designed and implemented via the PsiTurk platform (Gureckis et al., 2015), and coded in a combination of JavaScript (specifically, the JsPsych library; de Leeuw, 2015) and Python. The study took place over two one-hour sessions, separated by six to eight days (mean 6.7 days; median 6.9 days) to approximate the common time span between releases of subsequent television episodes. Participants ($N=70$) were divided into two equal groups. In the first session, all participants viewed the 25-minute pilot episode of the FX series, *Atlanta*. Following the episode, participants were then given up to an hour to recall its events, in order, to the best of their abilities. Mirroring Chen et al. (2017), they were asked to try to speak for a minimum of ten minutes and told that they were free to go back and describe earlier events, should they realize they missed any. Finally, they were given up to five minutes to predict what they thought would happen in the next episode. In the second session, all participants were again given up to one hour to recall to recall the episode viewed the prior week. Group one then viewed the second episode of *Atlanta*, while group two viewed the pilot of *Arrested Development*. Both groups finished by recalling the newly watched episode in the same manner. Eleven subjects' data were discarded for various reasons prior to processing and analysis: five did not follow the provided instructions, two did not complete the task, two did not return for the second session, one dropped out due to illness, and one was unable to complete the experiment due to technical issues. The data analyzed comprise the remaining 59 participants (30 in group one; 29 in group two).

2.2 Stimulus annotation and audio transcription

The ability of Heusser et al. (2018a)'s approach to model the explicit content of a naturalistic experience relies on the availability of detailed, hand-generated annotations that characterize the experience's evolving content. To create the annotation corpus for each television episode, I used ANVIL (Kipp, 2001), along with a custom XML specification file to record the following features for every camera shot:

- A brief description of physical occurrences and characters' actions ("*Narrative details: external events*")
- A brief description of characters' emotional states and various subtextual implications ("*Narrative details: internal states*")
- A list of characters appearing on-screen
- A transcription of all dialogue
- The name of the character speaking
- The shot's setting

- Whether the shot took place indoors or outdoors
- The presence or absence of music

These features were chosen both to be consistent with and to improve upon prior work with a similar set of stimulus annotation generated by Chen et al. (2017). Analyses carried out on Chen et al.’s feature set in conjunction with Heusser et al.’s modeling approach revealed that certain annotated features (e.g., what text appears on screen) contributed little-to-no structure to the stimulus trajectory. Further, some features (e.g., name of the character in focus and of the character speaking) were found to be highly redundant with each other, while others (e.g., the camera angle) were rarely preserved during recall and thus contributed unnecessary noise to the model. I chose to separate descriptions of physical actions from internal emotions and subtextual implications both to acknowledge that not all content is presented equally explicitly, and to enable future investigations by myself or others into potential differences in prioritization of those features in processing and memory.

All participant audio was transcribed using the Google Cloud Speech-to-Text API (Halpern et al., 2016). A previous proof-of-concept study (Ziman et al., 2018) suggested that this particular automatic transcriber may be used as an adequate stand-in for manual transcription of verbal response data, and as this data set amounted to approximately 50 hours of audio, I elected to use this approach. One significant asset to the Google transcription API is the option to bias the speech decoder toward certain words or phrase. For each audio file to be transcribed, I seeded the decoder with the union of words present in the hand-generated annotations of the reference episode (removing English stop-words, non-alphabetic characters, plurals, and possessives). Because Ziman et al. (2018) analyzed the quality of the Google transcriber in the context of a simple list-learning study, it was prudent to verify the decoding algorithm’s ability to return sufficiently accurate transcripts of natural language. Prior to transcribing the full data set, I tested the accuracy of the transcriber on a subset of audio files from five randomly chosen participants. I both automatically and manually transcribed each file, and constructed topic trajectories from each transcript method *insert href to section describing topic modeling procedure!!!!*. On average, the automatically and manually generated recall trajectories were similar across timepoints (Pearson’s $r=0.63$, $SEM=0.06$) and their similarities to the corresponding episode trajectory (a rough estimate of recall accuracy) differed by less than 2.5%.

2.3 Dynamic content modeling

Building off of Heusser et al. (2018a), I created a separate representational space corresponding to each episode. Using `Hypertools` (Heusser et al., 2018b) and its underlying `scikit-learn` (Pedregosa et al., 2011) implementation, I trained a topic model (Blei et al., 2003) on the collection of hand-generated annotations for each presented episode. This topic modeling approach gives rise to a high-dimensional space whose axes represent the latent themes (or “topics”) present in the collection of “documents” (in this case, annotations) upon which the model was trained. To create the document corpus for the topic models, I first concatenated the text describing all features within a single shot to create a multiset of words that captures the shot’s content. I then organized these “bags of words” into overlapping sliding windows spanning 50 consecutive shots, with each window offset from the previous by one (i.e., the first window encompassed shots 1-50, the second encompassed shots 2-51, etc.). The fact that the composition of each window changes by only a small fraction with each shift of its center reflects the well established theory that features of both the external world and individuals’ internal mental states tend to drift gradually over time (Howard and Kahana 2002; see Kahana 2012; Manning et al. 2015 for review).

Training a topic model on the resulting document corpus entails multiple steps. First, I fit an instance of `scikit-learn`’s `CountVectorizer` class to the union of words across all windows (excluding English stop-words). I used this vectorizer to subsequently transform the windows into a series of vectors describing the number of times each word appeared in each window. I then fit a `LatentDirichletAllocation` instance to these vectors, yielding a number-of-annotations by number-of-topics matrix. This matrix describes the trajectory of the episode’s evolving content through its self-defined representational space. Along this trajectory, the coordinates at each point reflect the weighted combination of topics present in the content at that point.

Within this framework, enabling comparison between the trajectory of an episode’s content and that of participants’ corresponding recalls or predictions requires that the trajectories share two features: the space in which they are defined and their temporal dimension. To facilitate the first, I similarly parsed participants’ transcripts into overlapping sliding windows (of 200 words) and used the `CountVectorizer` instance fit to the corresponding episode’s vocabulary to create word-count vectors. I then used the `LatentDirichletAllocation` instance already trained on that episode’s content to transform the recall/prediction windows into the *same* representational space as the reference episode. In order to standardize the lengths of a given episode and each participant’s recall/prediction for it, I interpolated the time series of the episode trajectory to a resolution of one second (using the time at the midpoint of each shot as a reference) and resampled all participant trajectories to the same length.

Note: as part of my topic modeling procedure, I chose to inherit various parameters from Heusser et al. (2018a), specifically the episode and recall sliding window lengths and the number of topics passed to the model. Heusser et al. performed a grid search over these parameters, choosing a combination that resulted in the strongest relationship between model- and human-generated memory quality metrics (50-annotation stimulus windows, 10-sentence recall windows, and 100 topics). As the decoding algorithm I used does not identify sentence breaks, I set my recall window size based on the average number of words in Heusser et al.’s recall windows.

2.4 Event segmentation, and video-recall mapping

Next, I used Hidden Markov models (Rabiner, 1989) to segment the continuous episode, recall, and prediction trajectories into K discrete “events.” Given a series of real-life events, the content within a tends to be highly self-similar (i.e., revolving around a specific set of people, actions, or ideas) and dissimilar to other, separate events. For each trajectory, I searched over a range of possible K -values (2-50) to identify the division that most likely corresponded to the episodes’ distinct scenes and participants’ descriptions of them. The optimal K -values were selected based on the maximum ratio of within-event content similarity to across-event content similarity, given by the expression

$$\operatorname{argmax}_K \left[\frac{a}{b} - \frac{K}{\alpha} \right].$$

Here, a represents the average correlation between topic vectors at timepoints within an event, b represents the average correlation between topic vectors across different events, and α represents a normalization term set to five times the window length (250 shots for episodes and 1000 words for recalls/predictions). The regularization parameter $\frac{K}{\alpha}$ is necessary to down-weight the priority of unrealistically high numbers of events (as the value of $\frac{a}{b}$ increases with shorter events) and was adopted from Heusser et al. (2018a).

I then estimated the content comprising each episode, recall, and prediction event by averaging over the topic vectors contained between the event boundaries. This allowed me to map between recalled/predicted content and presented content by finding the episode event to which each recall/prediction event was maximally similar, based on their average topic vectors. Importantly, these matches were computed in a non-exclusive way (i.e., multiple recall events were allowed to correspond to the same episode event), enabling me to account for participants who chose to go back further describe an early scene at a later point during the recall session.

2.5 Cortical map generation

To create the

3 Analyses to be employed and expected results

3.1 Comparing immediate and delayed recall

3.2 Measuring the effect of contextual familiarity on future memory success

3.3 Relating prediction ability to memory success

4 Implications

References

- Blei, D. M., Ng, A. Y., and Jordan, M. I. (2003). Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993 – 1022.
- Chen, J., Leong, Y. C., Norman, K. A., and Hasson, U. (2017). Shared experience, shared memory: a common structure for brain activity during naturalistic recall shared experience, shared memory: a common structure for brain activity during naturalistic recall. *Nature Neuroscience*, 20(1):115–125.
- de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a web browser. *Behavior Research Methods*, 47(1):1–12.
- Gureckis, T. M., Martin, J., McDonnell, J., Rich, A. S., Markant, D., Coenen, A., Halpern, D., Hamrick, J. B., and Chan, P. (2015). psiTurk: An open-source framework for conducting replicable behavioral experiments online. *Behavior Research Methods*, 48(3):829–842.
- Halpern, Y., Hall, K. B., Schogol, V., Riley, M., Roark, B., Skobeltsyn, G., and Bäuml, M. (2016). Contextual prediction models for speech recognition. In *Interspeech*, pages 2338–2342.
- Heusser, A. C., Fitzpatrick, P. C., and Manning, J. R. (2018a). How is experience transformed into memory? *bioRxiv*, 409987.
- Heusser, A. C., Ziman, K., Owen, L. L. W., and Manning, J. R. (2018b). HyperTools: a Python toolbox for gaining geometric insights into high-dimensional data. *Journal of Machine Learning Research*, 18(152):1–6.
- Howard, M. W. and Kahana, M. J. (2002). A distributed representation of temporal context. *Journal of Mathematical Psychology*, 46:269–299.
- Kahana, M. J. (2012). *Foundations of Human Memory*. Oxford University Press, New York, NY.
- Kipp, M. (2001). Anvil-a generic annotation tool for multimodal dialogue. In *Seventh European Conference on Speech Communication and Technology*.
- Manning, J. R., Norman, K. A., and Kahana, M. J. (2015). The role of context in episodic memory. In Gazzaniga, M., editor, *The Cognitive Neurosciences, Fifth edition*, pages 557–566. MIT Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Rabiner, L. (1989). A tutorial on Hidden Markov Models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286.
- Ziman, K., Heusser, A. C., Fitzpatrick, P. C., Field, C. E., and Manning, J. R. (2018). Is automatic speech-to-text transcription ready for use in psychological experiments? *Behavior Research Methods*, pages <https://doi.org/10.3758/s13428-018-1037-4>.